Instructor Handout 3: System Calls

Lecture 3 — Mechanics, Parameter Passing, and Types

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

3.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 2 "System Structures": §2.3 (system calls, pp. 55–58) and §2.4 (types of system calls, pp. 58–66). This is the primary anchor: the API/system-call distinction, the three parameter-passing methods, and the six-category taxonomy all come from these pages. *Page-verified* via the course index.
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 10 (system calls, pp. 398–419): POSIX APIs versus system calls, the handler and service routines, entering and exiting the kernel, parameter passing, and wrapper routines. Use it for concrete x86/Linux detail; do not use it for the taxonomy, which is Silberschatz's. *Page-verified.*
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 3 (process description and control, pp. 107–159), especially §3.4 on the context switch (p. 135), for the process context in which a system call executes. *Page-verified.*
- **Run the trace before class.** On Linux, `strace -e trace=openat,read,write,close cp notes.txt backup.txt` shows the real sequence. Expect more calls than the ten-call minimum: the dynamic loader opens shared objects before `main` runs. That discrepancy is a teaching opportunity, not a problem — the minimum counts the program's own work.
- **Draw Figure 3.1 from memory.** You must be able to place the trap, the table lookup, and the return arrow without looking. If the return path is fuzzy, reread Bovet & Cesati, Ch. 10, pp. 398–419.
- **Decide your concrete machine.** This handout assumes Linux x86-64: the `syscall` instruction, the number in `rax`, and arguments in `rdi`, `rsi`, `rdx`. If you teach a different ABI, substitute its registers consistently rather than mixing two.
- **Prerequisite check (two minutes).** Recall the mode bit, privileged instructions, the timer, and the user/kernel boundary from Lecture 1. Students do not need process scheduling yet; `fork`/`exec` are introduced here only as calls the shell makes.

Teaching note: The thesis to keep on the board all hour: a system call is a function call that changes privilege level, and the kernel validates everything that crosses. Every section either builds that sentence or uses it.

3.2 Opening: The Command That Keeps Knocking

Run, or simply say, `cp notes.txt backup.txt`, and ask the class what happens between pressing Enter and the file appearing. Four things happen in order: the program opens one file, creates another, copies every byte, and closes both. The prompt makes it look like one action; inside the machine it is a sequence of requests.

`cp` is ordinary user code. It cannot touch the disk, modify the page tables, or read another process's memory, because the hardware forbids those operations in user mode. Every privileged operation must be requested through a **system call**. Each request has four parts: a name, a number, a set of arguments, and a return value. The lecture follows those four parts across the boundary and back.

The roadmap is four questions. *What is a system call?* The kernel's programming interface. *How does control cross the boundary?* Through the trap and the system-call table. *How do arguments travel?* In registers, a memory block, or the stack. *What kinds of calls exist?* Six standard categories. The mechanism is hardware, the interface is software, and the taxonomy is the vocabulary the rest of the course uses.

Board: inventory the calls for a one-byte file. *On the board, in order:* (i) draw a horizontal line across the board and label the top *user mode*, the bottom *kernel mode*; (ii) list the calls down the left: `fork`, `exec`, `open`, `open`, `read`, `write`, `close`, `close`, `exit`, and the shell's `wait`; (iii) ask the class to guess the total before you count; (iv) write the count, ten. *Say:* "One command, ten requests, and every one of them is a separate conversation with the kernel."

Board: turn ten calls into crossings. *On the board, in order:* (i) keep the same list; (ii) draw one down-arrow and one up-arrow beside each call; (iii) count the arrows — ten down, ten up; (iv) write "ten round trips, twenty boundary crossings." *Say:* "A call is not one arrow. Entry is a crossing and return is a crossing; when the cost section says 'mode switch,' ask which of the two it means."

Teaching note: Do not reveal the number ten immediately. The guess is the hook: students who guess three or four are counting the *commands* they recognize (`open`, `read`, `write`), not the process-control calls around them. The gap between their guess and ten is the lecture's first correction.

Concepts That Click

3.3 API vs. System Call

Programmers rarely invoke a system call directly. They call a library function, and the library eventually executes the trap. That layering produces two views of one service, and Table 3.1 sets them side by side.

The left column is a convenience layer: it knows about buffering, hides the instruction set, and offers a uniform interface across operating systems. The right column is the actual request: a fixed number, a fixed calling convention, and a return value the kernel can produce. The columns are not in one-to-one correspondence. One library call may issue zero, one, or many system calls, because buffering changes the count.

Programmer's view (API)	Kernel's view (system call)
`fopen`, `fread`, `fwrite`, `fclose`	`open`, `read`, `write`, `close`
Standard C library; buffered; portable	Kernel entry points; unbuffered; architecture-specific
Returns a `FILE *` or a count	Returns a file descriptor or -1 with `errno`

Table 3.1: Two views of the same file service. The API is what the programmer writes; the system call is what the kernel executes. Silberschatz et al., Ch. 2 §2.3, pp. 55–58.

That is why `printf` is not a system call. It formats into a user-space buffer; when the buffer fills or is flushed, the library calls `write`, and `write` traps. Buffering exists to reduce expensive mode switches: twenty small pieces of output become one `write` and one crossing. The bytes delivered are the same; the number of crossings is not.

Why not link the kernel into every program and skip the boundary? Four properties would be destroyed. **Protection**: user code would run with kernel privileges and could corrupt the machine. **Isolation**: one buggy program could overwrite another's memory or the kernel's data. **Portability**: the same API can sit on different kernels only because the boundary hides the implementation. **Evolution**: the kernel can change its internals without recompiling every application. The trade-off is plain: every system call costs a mode switch, so the design goal is a *narrow* boundary and *efficient* calls.

Board: one hundred `printf` calls, one byte each. *On the board, in order:* (i) write "100 `printf`" on the left and "? `write`" on the right; (ii) draw the user-space buffer in the middle and mark it "4 KiB"; (iii) ask for a prediction — one write, one hundred, or a million; (iv) write the answer: *as few as one*, if the buffer does not fill and the program flushes once at exit. *Say:* "The output is one hundred bytes either way. What changed is how many times we paid for the door."

Board: one mebibyte read with `fgetc`. *On the board, in order:* (i) write "1,048,576 `fgetc` calls"; (ii) write "4 KiB buffer"; (iii) divide: $1{,}048{,}576/4096 = 256$ refills; (iv) conclude "about 256 `read` system calls, plus a final read that returns 0 if the loop reads to EOF." *Say:* "The program made a million function calls and roughly 256 boundary crossings. Both numbers are correct; they answer different questions."

> **Teaching note:** Students equate the API name with the system-call name because `write` appears in both columns. Point at the return type: a `FILE *` cannot come out of the kernel, and a file descriptor cannot come out of `fopen`. The types are the quickest proof that these are different layers.

Concepts That Click

3.4 What Exactly Is a System Call?

Definition (System call). *A **system call** is the programmatic request from a user program to a kernel service. It is the boundary of the operating system's programming interface.*

At the source level it looks like a function call: a name, parentheses, arguments, a return value. It is not an ordinary call, for three reasons. First, it *changes the privilege level*: the CPU leaves user mode, enters kernel mode, runs kernel code, and returns. Second, the kernel executes it *on behalf of the calling process* and *with the kernel's privileges*: the file belongs to the process, the bytes land in its buffer, but the code that acts is trusted kernel code. Third, the kernel *validates the request*

before acting: the user supplies the arguments, and the kernel assumes none of them are sensible. The third difference is what turns the boundary from a convenience into a security mechanism.

One warning about terminology. The same phrase names three layers: the API function, the trap instruction, and the kernel routine. Context decides which is meant, and conflating them causes most of the misconceptions below.

Board: ordinary call versus system call. *On the board, in order:* (i) draw two columns, "ordinary function call" and "system call"; (ii) under the first write *same privilege*, *callee is your code*, *no validation*, *plain return*; (iii) under the second write *privilege changes*, *callee is the kernel*, *arguments validated*, *return via saved PC*; (iv) circle "privilege changes" and "arguments validated." *Say:* "The syntax is the same. The rights are not."

Board: four phrases, three layers. *On the board, in order:* (i) write `printf`, `write`, the `syscall` instruction, and "the kernel's write handler"; (ii) draw three boxes labeled *API*, *trap*, *kernel routine*; (iii) place `printf` in API, the instruction in trap, the handler in kernel routine, and `write` in *both* API and kernel routine; (iv) ask why `write` appears twice. *Say:* "One word, two layers. The context decides which one you mean."

> **Teaching note:** Ask: "Is `write` an API or a system call?" The correct answer is "both names, different layers." Let the ambiguity sit for a few seconds; students who rush to one answer are the ones who later say the kernel can trust a user pointer.

3.5 The Trap Path

The trap instruction is the hinge of the whole mechanism: one instruction moves the CPU from user mode to kernel mode and hands the kernel a number. Figure 3.1 shows the path that instruction starts.

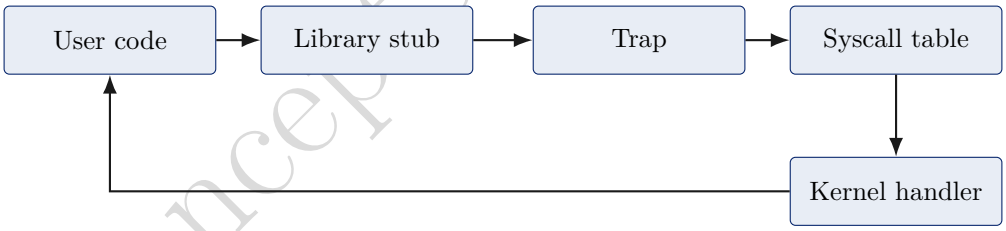


Figure 3.1: The trap path. User code calls a library stub; the stub executes the trap; the kernel indexes its table by number, runs the handler, and returns to the user program. Silberschatz et al., Ch. 2 §2.3, pp. 55–58; concrete Linux detail in Bovet & Cesati, Ch. 10, pp. 398–419.

The path has three stages. In the first, the library stub places the **system-call number** and the arguments in registers. The number identifies the requested service; the registers are where the kernel will look. In the second, the trap instruction saves the user program counter and flags, switches the CPU to kernel mode, and jumps to the kernel's trap entry point. Saving the PC is what makes a return possible: the kernel must resume at the instruction after the trap. In the third, the kernel indexes the **system-call table** by the number it received and calls the corresponding handler. When the handler finishes, the kernel restores the user context and returns.

Build the figure left to right, narrating the boundary as you go. Start with *user code*; add the *library stub* and say both still run in user mode. Add the *trap* and draw a vertical line between stub and trap: the instruction *begins* in user mode and *completes* in kernel mode, which is why the

Concepts That Click

line belongs there. Add the *table*, the *handler* below it, and the return arrow along the bottom. Say explicitly that the arrows form a path, not a cycle: the user program does not run while the kernel works.

Board: reproduce Figure 3.1 from memory. *On the board, in order:* (i) five boxes in the same positions; (ii) forward arrows user → stub → trap → table → handler; (iii) the return arrow from handler left along the bottom and up into user code; (iv) label the mode boundary at the trap. *Say:* "If you can draw this without the sheet, you can explain any system call in the course."

Board: the seven-step `write(fd, buf, n)` trace. *On the board, in order:* (i) number the steps 1–7 down the left margin; (ii) write each step in a few words; (iii) draw a bracket marking steps 1–2 as user mode, 3–6 as kernel, 7 as user again; (iv) count the round trip and circle step 5, validation. The steps: (1) `fd`, `buf`, `n` go into argument registers; (2) the number for `write` goes into its designated register; (3) the stub executes `syscall` (x86-64) or `int 0x80` (legacy x86); (4) hardware switches to kernel mode, saves the return address, enters the trap handler; (5) the kernel validates `fd`, checks that `buf` lies in user memory, and performs the write; (6) the return value goes into a register, user mode is restored, and the call returns; (7) on error, the library sets `errno` and returns -1. *Say:* "Steps 1–2 are still your program. Steps 3–6 are the kernel. Step 7 is your program again, reading a value it must check."

> **Teaching note:** The most common error at this point is to think the user program "waits inside the call" while the kernel runs something else. It does wait, in the sense that its next instruction has not executed — but the CPU is executing kernel code, not the user's code. The saved PC is the entire reason the program can continue afterward. Draw the PC being pushed onto the kernel stack; that single detail fixes the confusion.

Concepts That Click

3.6 The System-Call Table, ABI, and `errno`

Definition (System-call table). *A kernel array indexed by system-call number. Each entry is a function pointer to the handler for that service.*

The trap carries only a number, not a name or a user-chosen address. The table turns the number into the right routine without a search: the number is an index, so the kernel goes directly to the entry. Four properties follow. The number is the **ABI** (application binary interface) and must stay stable across kernel versions, because a compiled program contains the number, not the name. Adding a call means adding a table entry and a handler, not changing the trap instruction. The handler runs in kernel mode with the kernel's privileges, which is why validation is possible at all. And dispatch is a lookup rather than a search. On Linux x86-64, `write` is system call 1, and the table lives in the kernel's trap entry code.

A system call reports three outcomes: success with a value, success with no data, and failure. The kernel returns a value in a register — a count, a descriptor, or a negative error code. The library wrapper translates a negative value into -1 and sets `errno`. The caller must check the return value; the kernel cannot print a message for it.

Board: dispatch by number. *On the board, in order:* (i) draw a vertical array with indices $0, 1, 2, \ldots, N$; (ii) put "`sys_write`" in slot 1 and "…" in the others; (iii) draw the trap box on the left with an arrow labeled "1" into slot 1; (iv) draw the arrow out to the handler. *Say:* "No name is sent. One integer, one indexed load, one handler. If the user sent a name, the kernel would have to search a string the user controls."

Board: the three outcomes of `read`. *On the board, in order:* (i) three columns: "value", "meaning", "program action"; (ii) row 1: $n > 0$, "n bytes read", "consume them"; (iii) row 2: 0,

"end of file", "stop, not an error"; (iv) row 3: -1, "error; `errno` set", "inspect `errno`"; (v) write `EFAULT` and `EBADF` under row 3. *Say:* "End of file is a successful call that moved zero bytes. If you treat 0 as failure, you will report an error on every normal file."

> **Teaching note:** Correct the sentence "the kernel returns -1." The kernel returns a negative error code; the *library wrapper* converts it to -1 and sets `errno`. The distinction matters when a student later reads kernel code and finds no `-1` anywhere in the handler.

3.7 How Arguments Travel

A system call is a function call across a privilege boundary. The arguments must cross too, and the kernel must not trust any of them. Figure 3.2 arranges the three standard mechanisms by their trade-off.

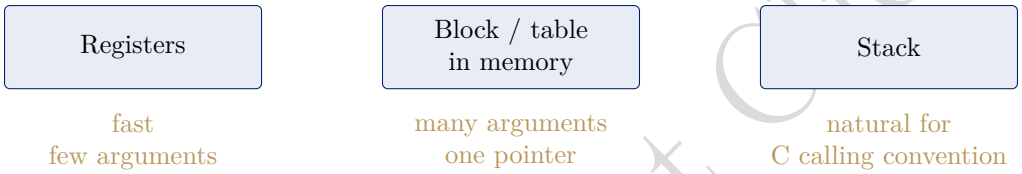


Figure 3.2: The three standard mechanisms for passing system-call arguments. Silberschatz et al. describe all three in Ch. 2 §2.3, pp. 55–58.

Registers are fastest: the values are already in the CPU, so no memory access is needed beyond the registers saved on entry. The limit is the number of argument registers. **A block or table in memory** stores the arguments in a block and passes the block's address in one register; one register now carries an arbitrary number of arguments, at the price of one indirection and a copy. The **stack** is natural for conventions that already pass arguments there: the arguments are where the kernel expects them, so the stub need not rearrange anything. Real ABIs choose one or a mix.

Whichever mechanism is used, the kernel must do the same three things: *locate* the arguments, *copy* them into kernel memory so that a user program cannot change them underneath the kernel, and *validate* every pointer and length before using it. The mechanisms differ in where the arguments start; they do not differ in what the kernel must do once it finds them.

Board: the register map for `read(fd, buf, count)`. *On the board, in order:* (i) draw a four-row table with columns "register" and "contents"; (ii) fill `rax` ← "number for `read`", `rdi` ← `fd`, `rsi` ← `buf`, `rdx` ← `count`; (iii) instantiate it: `fd` = 3, `buf` = `0x7ffc1000`, `count` = 4096; (iv) draw an arrow from `rsi` to a box labeled "user buffer"; (v) circle the word *validate* beside the arrow. *Say:* "`rsi` says where the buffer is. It does not say the process is allowed to use it."

Board: a seven-argument call passed as a block. *On the board, in order:* (i) draw a seven-cell block labeled $a_1, \ldots, a_7$; (ii) draw one register holding the block's base address; (iii) draw an arrow from the register to the first cell and a bracket over all seven cells; (iv) annotate "copy all seven into kernel memory, then validate each pointer and length"; (v) draw a second block where the first three cells are in user space and the last four in kernel space, and put a large cross through it. *Say:* "The pointer is only the beginning. The kernel must prove the whole block is yours before it reads a single field."

> **Teaching note:** The copy is not decoration. A user program can change its buffer while the kernel is working — another thread, a signal handler, or a second process sharing the mapping.

Concepts That Click

Copying into kernel memory gives the kernel a stable snapshot; validating before copying gives it a safe source. Ask which of the three mechanisms skips the copy: none of them may.

3.8 Pointer Validation: Never Trust User Memory

A user pointer is just a number, and it may point anywhere: into kernel memory, into another process's memory, or into unmapped memory that will fault on access. The kernel therefore checks the range against the process's address space before it uses the pointer. It then copies with special routines (`copy_from_user` and `copy_to_user` on Linux) that handle faults safely. A bad pointer returns `EFAULT`; it does not crash the kernel.

The security rule is absolute: *every argument is untrusted input*, and validation happens in kernel mode, before the kernel acts. If the kernel trusted a user pointer, a program could pass the address of a kernel data structure to `read` and receive kernel memory as output, or pass it to `write` and overwrite kernel state. Dereferencing directly would make the protection of the previous sections meaningless.

Board: the kernel-address attack. *On the board, in order:* (i) draw three horizontal regions: kernel, process A, process B; (ii) put a "user program" box inside process A and a "kernel data" box inside the kernel region; (iii) draw a pointer arrow from the user program straight into kernel data and label it "`buf`"; (iv) draw a large cross over the arrow and write `EFAULT` beside it; (v) draw the correct path: a range check, then a copy into a kernel buffer. *Say:* "The dangerous move is not passing the number. The dangerous move is following it."

Board: a block that straddles the boundary. *On the board, in order:* (i) draw the user/kernel line; (ii) draw a block of six fields starting above the line and crossing below it; (iii) mark the first three fields "valid" and the last three "kernel"; (iv) write "reject the whole block" and `EFAULT`. *Say:* "Checking only the first byte would let the kernel read its own memory through the back half of the block."

> **Teaching note:** Students hear `EFAULT` and assume something crashed. It is the opposite: `EFAULT` is the kernel reporting that the request was invalid *without* touching the address. The check is not an optimization; it is the boundary being enforced.

3.9 The Six Categories

System calls are not a random list. They fall into six categories, and every command you run is a combination of them. Table 3.2 gives the taxonomy.

The categories are not a hierarchy, and a single operation may touch several. They are a vocabulary for describing what a call is *for*.

Process control manages a process's life cycle. `fork` creates a child that is a copy of the parent; `exec` replaces the current image with a new program; `wait` blocks until a child terminates and collects its status; `exit` terminates the caller and releases its resources. The shell is the canonical combination: it `fork`s, the child `exec`s the command, and the parent `wait`s. When you type `cp notes.txt backup.txt`, `cp` is the child after `exec`, and the prompt returns only when the copy finishes.

File management covers files and the descriptors that represent them: `open` returns a descriptor; `read`/`write` transfer bytes; `lseek` moves the offset without transferring; `close` releases the descriptor; `stat` reads metadata. A **file descriptor** is a small integer index into the process's

Concepts That Click

Category	Purpose	Typical calls
Process control	create, terminate, wait, load	`fork`, `exec`, `wait`, `exit`
File management	create, open, read, write, close	`open`, `read`, `write`, `close`, `lseek`
Device management	request/release, read/write, control	`ioctl`, `read`, `write`
Information maintenance	get/set time, process, file attributes	`getpid`, `alarm`, `sleep`, `stat`
Communication	create/delete connections, send/receive	`pipe`, `mmap`, `socket`, `send`, `recv`
Protection	set/get permissions	`chmod`, `umask`, `chown`, `setuid`

Table 3.2: The six categories of system calls. Silberschatz et al., Ch. 2 §2.4, pp. 58–66.

per-process open-file table — not the file itself. The descriptor is a handle meaningful only in the process that owns it; the table entry behind it is what refers to the file.

Device management uses `ioctl` for device-specific control and `read`/`write` for data. Devices are often accessed through the same descriptor interface as files, which is why the same calls appear in two rows. **Information maintenance** queries or sets small pieces of state: `getpid`, `alarm`, `sleep`, `time`, and `stat`. **Communication** moves data between processes or across a network: `pipe`, `mmap`, `socket`, `send`, `recv`. **Protection** changes who is allowed to do what: `chmod`, `umask`, `chown`, `setuid`. These calls do not read or write file contents; they change the rights to do so.

Board: classify six calls. *On the board, in order:* (i) write the six category names as column headers; (ii) list `fork`, `open`, `ioctl`, `getpid`, `pipe`, `chmod` down the side; (iii) ask the class to place each one before you write it; (iv) fill the table. *Say:* "Classify the operation, not the resource it happens to touch."

Board: `fork` versus file management. *On the board, in order:* (i) write the challenge: "`fork` creates a child that may open files — why is it not file management?"; (ii) write "operation: create a process" beside `fork`; (iii) write "inherits the open-file table, but opens nothing"; (iv) conclude "process control." *Say:* "The child's future file is not the call's purpose. If you classify by consequences, every call becomes every category."

> **Teaching note:** Use `ioctl` as the second classification trap. It takes a file descriptor, so students want to file it under file management; it is device management because its purpose is device-specific control. The descriptor is the *door*, not the *room*.

3.10 The Running Thread Across the Categories

The categories stop being abstract when the running thread is laid out against them. The shell `fork`s and the child `exec`s `cp`; `cp` opens the source and the destination; a `read`/`write` loop moves the bytes; both are closed; `exit(0)` ends the child and the shell's `wait` collects the status.

For a one-byte file that is ten calls: one `fork`, one `exec`, two `open`, one `read`, one `write`, two `close`, one `exit`, one `wait`. Eight of them happen before any data moves; the read and write bring the minimum to ten. A real file repeats the read/write pair, so the count grows with the number of chunks, not with the number of commands. This is the answer to the question the lecture opened with: `cp` crosses the boundary at least ten times for a one-byte file, and what it says each time is a number followed by arguments the kernel validates.

Concepts That Click

Board: annotate the `cp` trace. *On the board, in order:* (i) write the five steps in a left column; (ii) write the category beside each step in a middle column; (iii) write the call count in a right column and total it; (iv) circle the four file-management calls and the four process-control calls. *Say:* "Two categories do all the work. The other four are waiting for a command that needs them."

Board: scale the trace to a 4 MiB file with a 4 KiB buffer. *On the board, in order:* (i) write "4 MiB = 4096 KiB"; (ii) write "4096/4 = 1024 chunks"; (iii) write "1024 `read` + 1024 `write` = 2048 calls"; (iv) add the eight fixed calls and write "2056"; (v) ask what happens if the buffer doubles. *Say:* "The fixed calls are a rounding error. The loop is the cost, and the loop is measured in chunks."

> **Teaching note:** Ask the class to predict the count before the arithmetic: most will double the ten and say twenty. The correct answer is in the thousands, because they are still thinking in commands while the machine is thinking in buffers. The prediction failure is the lesson.

3.11 Cost, ABI, and Compatibility

A system call is not free. The cost has four components: save the user context and switch to kernel mode; index the table and enter the handler; validate the arguments and copy data across the boundary; perform the operation and restore the context. Each is small, but the copy is proportional to the data moved, and the switch is paid per call. That is why batching matters.

The interface is stable for a reason. Each architecture defines a number for each service, and the numbers are part of the ABI. A kernel cannot renumber existing calls without breaking every compiled program, because the number is embedded in the binary, not looked up by name at run time. New calls are added at the end of the table; old calls remain for compatibility. The consequence is that the table is a long-lived contract between the kernel and every binary that runs on it: it can grow, but it cannot be silently rewritten.

Board: one mebibyte, one byte at a time. *On the board, in order:* (i) write "1,048,576 bytes"; (ii) write "1-byte chunks: 1,048,576 `read` + 1,048,576 `write` = 2,097,152 calls"; (iii) write "one big read and one big write: 2 calls"; (iv) divide: the ratio is 1,048,576; (v) write "same bytes, different number of doors." *Say:* "The bytes did not get cheaper. The crossings did."

Board: the renumbering thought experiment. *On the board, in order:* (i) draw a binary box containing "`write` = 1"; (ii) draw kernel table A with `sys_write` in slot 1 and the call succeeding; (iii) draw kernel table B with `sys_write` moved to slot 7 and slot 1 reused; (iv) draw the same binary calling 1 and landing in the wrong handler; (v) write "ABI break." *Say:* "This is why new calls go at the end. The table is append-only history."

> **Teaching note:** Separate the two costs explicitly on the board: a *fixed* cost per crossing and a *per-byte* cost for the copy. Batching reduces the first and does nothing for the second. That is why one huge transfer is not automatically better — it trades crossings for memory, and the per-byte copy still happens.

3.12 Exit Ticket / Comprehension Checks

Use the three Socratic Checks from the deck, in order, then the two calculations.

1. `printf` is not a system call. What does it eventually call, and why does the library buffer first? *Answer:* `write`; formatting happens in a user-space buffer, and buffering reduces the number of mode switches.

Concepts That Click

2. A user program passes a pointer to a kernel address to `read`. What should happen, and why can the kernel not dereference it? *Answer:* the call fails with `EFAULT`; dereferencing would let user code read or overwrite kernel data.

3. Which category does `fork` belong to, and why is it not file management? *Answer:* process control; it creates a process and opens no file, even though the child inherits the open-file table. Classify the operation, not its consequences.

4. `cp` on a one-byte file issues ten calls. How many round trips, and how many boundary crossings? *Answer:* ten round trips; twenty crossings if entry and return are counted separately.

5. A 4 MiB copy in 4 KiB chunks issues how many calls, and how many with one large read and one large write? *Answer:* 2048 data calls (plus the fixed process-control calls), versus 2 data calls.

3.13 Summary: One-Page Refresh

- A **system call** is a program's request to the kernel, invoked through a trap, not an ordinary call. It changes privilege level, runs kernel code on the caller's behalf, and validates the request.
- The **system-call number** selects a handler from the kernel's dispatch table. The number is the ABI; the table is append-only across kernel versions.
- Arguments travel in **registers** (fast, few), a **memory block** (many, one pointer), or the **stack** (natural for the C convention). The kernel locates, copies, and validates every argument.
- Every user pointer and length is untrusted. A bad pointer yields `EFAULT`, not a kernel crash.
- The six categories are process control, file management, device management, information maintenance, communication, and protection. Classify the operation, not the resource.
- `cp notes.txt backup.txt` is at least ten calls for a one-byte file: `fork`, `exec`, two `open`, `read`, `write`, two `close`, `exit`, and the shell's `wait`. The count grows with chunks, not commands.
- Mode switches are expensive. The library API buffers to reduce them; batching reduces the fixed cost but not the per-byte copy.

3.14 Readings to Prepare Next Time

- **Silberschatz et al., *Operating System Concepts*, 8e, Ch. 2 §2.5–2.8** — system programs (p. 66), design and implementation, the five structures: monolithic, layered, microkernel, modular (pp. 70–76), virtual machines (p. 76), and system boot (p. 89). *(Page-verified.)* This is the direct setup for the next lecture.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e, Ch. 4 §4.3** — microkernels (p. 179), for the structural contrast with a monolithic kernel. *(Page-verified.)*
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e, Ch. 1 (pp. 1–34)** — optional: the Linux kernel as a monolithic kernel with loadable modules, the concrete counterpart to the taxonomy. *(Page-verified.)*

Concepts That Click